

sequence of n INCREMENT and RESET operations takes $O(n)$ time on an initially zero counter. (*Hint*: Keep a pointer to the high-order 1.)

16.3 The potential method

Instead of representing prepaid work as credit stored with specific objects in the data structure, the *potential method* of amortized analysis represents the prepaid work as “potential energy,” or just “potential,” which can be released to pay for future operations. The potential applies to the data structure as a whole rather than to specific objects within the data structure.

The potential method works as follows. Starting with an initial data structure D_0 , a sequence of n operations occurs. For each $i = 1, 2, \dots, n$, let c_i be the actual cost of the i th operation and D_i be the data structure that results after applying the i th operation to data structure D_{i-1} . A *potential function* Φ maps each data structure D_i to a real number $\Phi(D_i)$, which is the *potential* associated with D_i . The *amortized cost* $\hat{c}_i$ of the i th operation with respect to potential function Φ is defined by

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) . \quad (16.2)$$

The amortized cost of each operation is therefore its actual cost plus the change in potential due to the operation. By equation (16.2), the total amortized cost of the n operations is

$$\begin{aligned} \sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) . \end{aligned} \quad (16.3)$$

The second equation follows from equation (A.12) on page 1143 because the $\Phi(D_i)$ terms telescope.

If you can define a potential function Φ so that $\Phi(D_n) \geq \Phi(D_0)$, then the total amortized cost $\sum_{i=1}^n \hat{c}_i$ gives an upper bound on the total actual cost $\sum_{i=1}^n c_i$. In practice, you don’t always know how many

operations might be performed. Therefore, if you require that $\Phi(D_i) \geq \Phi(D_0)$ for all i , then you guarantee, as in the accounting method, that you've paid in advance. It's usually simplest to just define $\Phi(D_0)$ to be 0 and then show that $\Phi(D_i) \geq 0$ for all i . (See Exercise 16.3-1 for an easy way to handle cases in which $\Phi(D_0) \neq 0$.)

Intuitively, if the potential difference $\Phi(D_i) - \Phi(D_{i-1})$ of the i th operation is positive, then the amortized cost $\hat{C}_i$ represents an overcharge to the i th operation, and the potential of the data structure increases. If the potential difference is negative, then the amortized cost represents an undercharge to the i th operation, and the decrease in the potential pays for the actual cost of the operation.

The amortized costs defined by equations (16.2) and (16.3) depend on the choice of the potential function Φ . Different potential functions may yield different amortized costs, yet still be upper bounds on the actual costs. You will often find trade-offs that you can make in choosing a potential function. The best potential function to use depends on the desired time bounds.

Stack operations

To illustrate the potential method, we return once again to the example of the stack operations PUSH, POP, and MULTIPOP. We define the potential function Φ on a stack to be the number of objects in the stack. The potential of the empty initial stack D_0 is $\Phi(D_0) = 0$. Since the number of objects in the stack is never negative, the stack D_i that results after the i th operation has nonnegative potential, and thus

$$\begin{aligned}\Phi(D_i) &\geq 0 \\ &= \Phi(D_0).\end{aligned}$$

The total amortized cost of n operations with respect to Φ therefore represents an upper bound on the actual cost.

Now let's compute the amortized costs of the various stack operations. If the i th operation on a stack containing s objects is a

PUSH operation, then the potential difference is

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &= (s + 1) - s \\ &= 1.\end{aligned}$$

By equation (16.2), the amortized cost of this PUSH operation is

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + 1 \\ &= 2.\end{aligned}$$

Suppose that the i th operation on the stack of s objects is MULTIPOP(S, k), which causes $k' = \min \{s, k\}$ objects to be popped off the stack. The actual cost of the operation is k' , and the potential difference is

$$\Phi(D_i) - \Phi(D_{i-1}) = -k'.$$

Thus, the amortized cost of the MULTIPOP operation is

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0.\end{aligned}$$

Similarly, the amortized cost of an ordinary POP operation is 0.

The amortized cost of each of the three operations is $O(1)$, and thus the total amortized cost of a sequence of n operations is $O(n)$. Since $\Phi(D_i) \geq \Phi(D_0)$, the total amortized cost of n operations is an upper bound on the total actual cost. The worst-case cost of n operations is therefore $O(n)$.

Incrementing a binary counter

As another example of the potential method, we revisit incrementing a k -bit binary counter. This time, the potential of the counter after the i th INCREMENT operation is defined to be the number of 1-bits in the counter after the i th operation, which we'll denote by b_i .

Here is how to compute the amortized cost of an INCREMENT operation. Suppose that the i th INCREMENT operation resets t_i bits to 0. The actual cost c_i of the operation is therefore at most $t_i + 1$, since in addition to resetting t_i bits, it sets at most one bit to 1. If $b_i = 0$, then the i th operation had reset all k bits to 0, and so $b_{i-1} = t_i = k$. If $b_i > 0$, then $b_i = b_{i-1} - t_i + 1$. In either case, $b_i \leq b_{i-1} - t_i + 1$, and the potential difference is

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i.\end{aligned}$$

The amortized cost is therefore

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leq (t_i + 1) + (1 - t_i) \\ &= 2.\end{aligned}$$

If the counter starts at 0, then $\Phi(D_0) = 0$. Since $\Phi(D_i) \geq 0$ for all i , the total amortized cost of a sequence of n INCREMENT operations is an upper bound on the total actual cost, and so the worst-case cost of n INCREMENT operations is $O(n)$.

The potential method provides a simple and clever way to analyze the counter even when it does not start at 0. The counter starts with b_0 1-bits, and after n INCREMENT operations it has b_n 1-bits, where $0 \leq b_0, b_n \leq k$. Rewrite equation (16.3) as

$$\sum_{i=1}^n c_i = \sum_{i=1}^n \hat{c}_i - \Phi(D_n) + \Phi(D_0).$$

Since $\Phi(D_0) = b_0$, $\Phi(D_n) = b_n$, and $\hat{c}_i \leq 2$ for all $1 \leq i \leq n$, the total actual cost of n INCREMENT operations is

$$\begin{aligned}\sum_{i=1}^n c_i &\leq \sum_{i=1}^n 2 - b_n + b_0 \\ &= 2n - b_n + b_0.\end{aligned}$$

In particular, $b_0 \leq k$ means that as long as $k = O(n)$, the total actual cost is $O(n)$. In other words, if at least $n = \Omega(k)$ INCREMENT operations occur, the total actual cost is $O(n)$, no matter what initial value the counter contains.

Exercises

16.3-1

Suppose you have a potential function Φ such that $\Phi(D_i) \geq \Phi(D_0)$ for all i , but $\Phi(D_0) \neq 0$. Show that there exists a potential function Φ' such that $\Phi'(D_0) = 0$, $\Phi'(D_i) \geq 0$ for all $i \geq 1$, and the amortized costs using Φ' are the same as the amortized costs using Φ .

16.3-2

Redo Exercise 16.1-3 using a potential method of analysis.

16.3-3

Consider an ordinary binary min-heap data structure supporting the instructions INSERT and EXTRACT-MIN that, when there are n items in the heap, implements each operation in $O(\lg n)$ worst-case time. Give a potential function Φ such that the amortized cost of INSERT is $O(\lg n)$ and the amortized cost of EXTRACT-MIN is $O(1)$, and show that your potential function yields these amortized time bounds. Note that in the analysis, n is the number of items currently in the heap, and you do not know a bound on the maximum number of items that can ever be stored in the heap.

16.3-4

What is the total cost of executing n of the stack operations PUSH, POP, and MULTIPOP, assuming that the stack begins with s_0 objects and finishes with s_n objects?

16.3-5

Show how to implement a queue with two ordinary stacks (Exercise 10.1-7) so that the amortized cost of each ENQUEUE and each DEQUEUE operation is $O(1)$.

16.3-6

Design a data structure to support the following two operations for a dynamic multiset S of integers, which allows duplicate values:

INSERT(S, x) inserts x into S .

DELETE-LARGER-HALF(S) deletes the largest $\lceil |S|/2 \rceil$ elements from S .

Explain how to implement this data structure so that any sequence of m INSERT and DELETE-LARGER-HALF operations runs in $O(m)$ time. Your implementation should also include a way to output the elements of S in $O(|S|)$ time.

16.4 Dynamic tables

When you design an application that uses a table, you do not always know in advance how many items the table will hold. You might allocate space for the table, only to find out later that it is not enough. The program must then reallocate the table with a larger size and copy all items stored in the original table over into the new, larger table. Similarly, if many items have been deleted from the table, it might be worthwhile to reallocate the table with a smaller size. This section studies this problem of dynamically expanding and contracting a table. Amortized analyses will show that the amortized cost of insertion and deletion is only $O(1)$, even though the actual cost of an operation is large when it triggers an expansion or a contraction. Moreover, you'll see how to guarantee that the unused space in a dynamic table never exceeds a constant fraction of the total space.

Let's assume that the dynamic table supports the operations TABLE-INSERT and TABLE-DELETE. TABLE-INSERT inserts into the table an item that occupies a single *slot*, that is, a space for one item. Likewise, TABLE-DELETE removes an item from the table, thereby freeing a slot. The details of the data-structuring method used to organize the table are unimportant: it could be a stack (Section 10.1.3), a heap (Chapter 6), a hash table (Chapter 11), or something else.